线上赛任务三报告

1. 技术任务要求
2. 运行环境
3. 原理与代码分析
   3.1 功能简介
   3.2 透视变换
   3.3 HSV转换
   3.4 HSV选通
   3.5 平滑边缘
   3.6 筛选车道线
      3.6.1 轮廓检测
      3.6.2 筛选有效点
         3.6.2.1 按区域大小筛选
         3.6.2.2 多边形拟合与按边数筛选
         3.6.2.3 (可选)按长宽比筛选
         3.6.2.4 存储各轮廓重心
   3.7 连通性判定
      3.7.1 寻找车道起点
      3.7.2 迭代剔除二维数据离群点
   3.8 展示输出
      输出车道线中点坐标
      绘制车道线
4.创新点分析
5. 成果展示

1. 技术任务要求

- 使用OpenCV、深度学习等方法（自选），识别提供视频中的车道线
- 输出车道线中点在每帧图像中的坐标数组

哈工大威海创新平台 - 718-王硕(22969079)

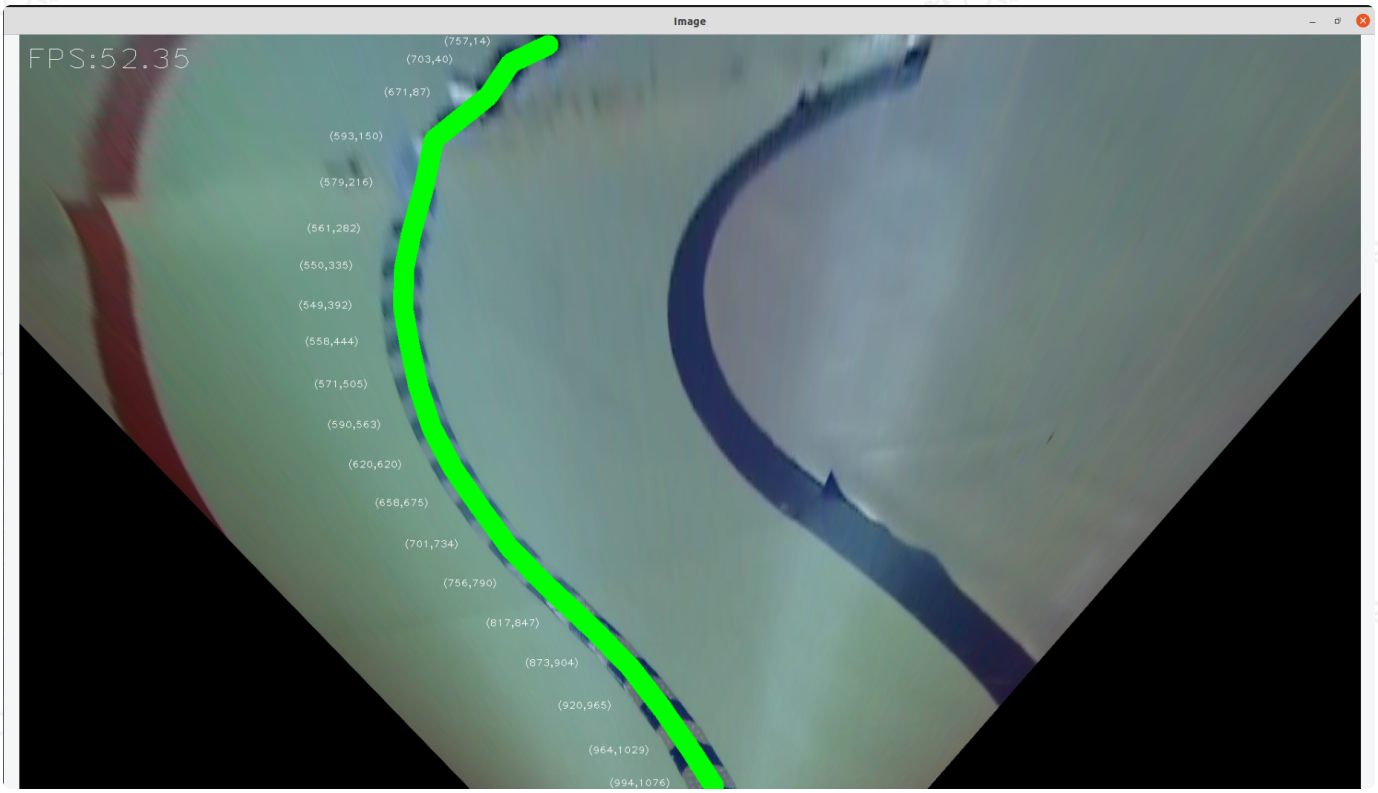
2. 运行环境

本任务较为简单，使用**传统视觉**方式即可完成，在**Ubuntu20.04**中运行，使用的编程语言为**C语言**，CMake版本为**3.21.1**，opencv版本为**4.5.3**

3. 原理与代码分析

3.1 功能简介

本工程将读取的视频逐帧读取并处理，经**透视变换**后按照设定的筛选标准**筛选**出车道线上的**蓝色色块**并求其**重心**，最后使用一根较粗的线连接起来来标示车道线，同时输出各重心坐标在图片上并**逐帧展示**



3.2 透视变换

此处本来应该是使用**内参**和**外参矩阵**来进行变换，但由于没有实车无法标定，我们找到了**去年**提供的参数（如下图），对今年的视频**抽帧**（抽帧的程序为工程文件中的**1.cpp**）并进行测试

打开(O) _01WarpMain.py ~/Desktop/03.计算机视觉/源码/深度学习-图像分割训练 保存(S)

```python
# -*- coding:utf-8 _*-
"""
@Author  : Yu Cheng
@Time    : 2019/12/25 9:35
"""
import glob, cv2, os
import numpy as np
import matplotlib.pyplot as plt

np.set_printoptions(suppress=True, precision=4)

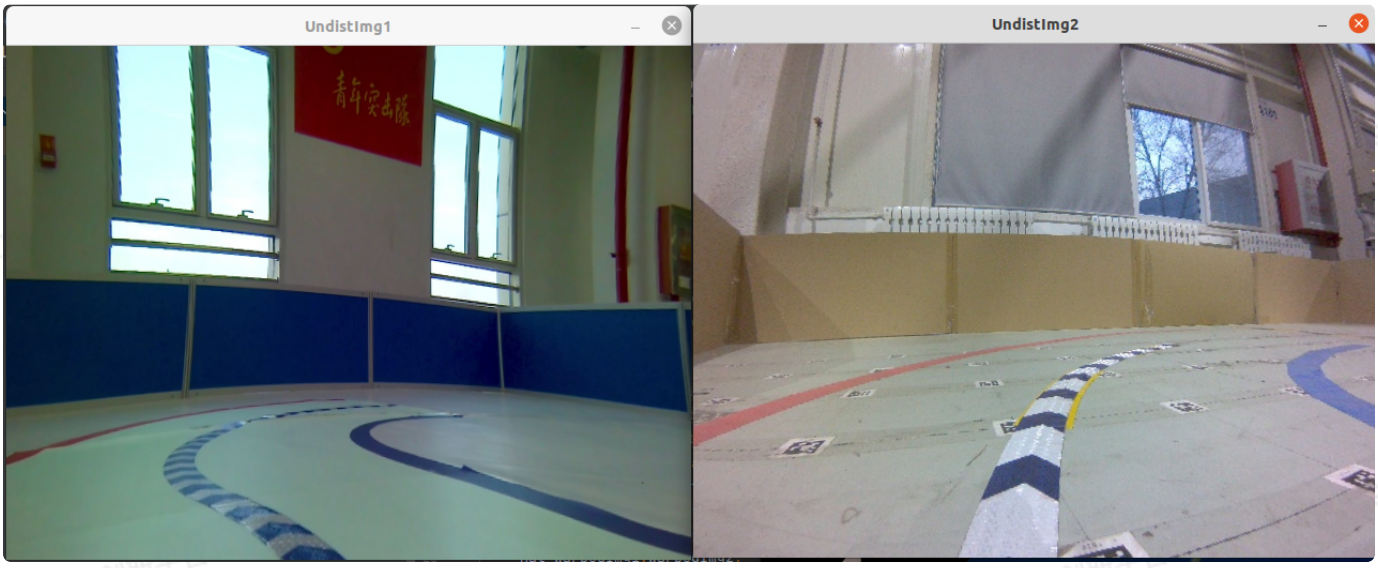
ImgPaths = glob.glob("OriImage\\*.jpg")
H = np.array([[  -0.61869793,   -2.24344654,  672.50410257],
              [   0.00583877,    0.05218149, -226.93620917],
              [  -0.00011433,   -0.00451613,    1.        ]])

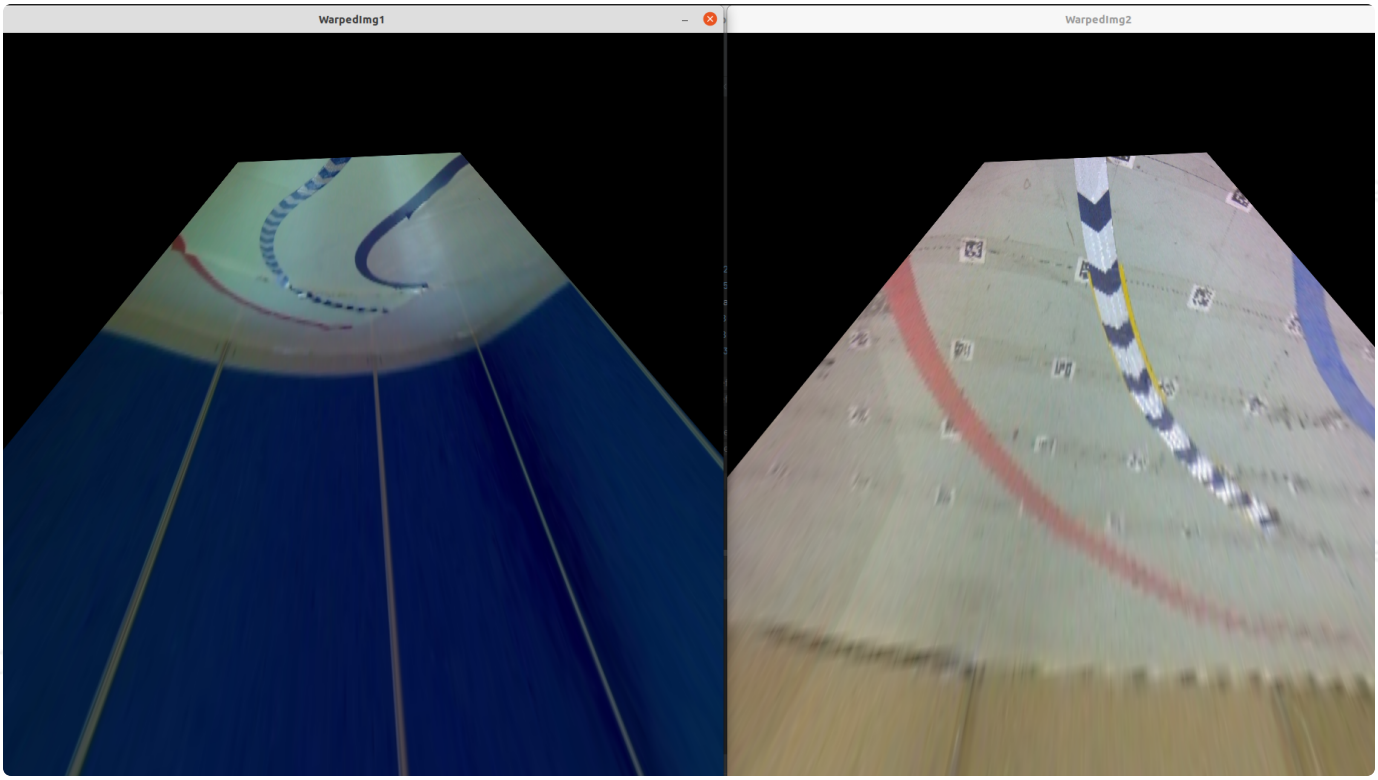
# 相机参数
Dist = np.array([-0.26538, 0.08153, -0.00109, -0.00233, 0.00000], dtype=np.float32)
K = np.array([[331.71415, 0, 321.54719],
              [0, 331.80738, 201.23948],
              [0, 0, 1]], dtype=np.float32)

for ImgPath in ImgPaths:
        print(ImgPath)
        Img = cv2.imread(ImgPath)
        UndistImg = cv2.undistort(Img, K, Dist)
        WarpedImg = cv2.warpPerspective(UndistImg, H, (1000, 1000))
        # plt.imshow(WarpedImg)
        #       # plt.show()
        SavePath = ImgPath.replace('OriImage', 'WarpedImg')
        os.makedirs(os.path.dirname(SavePath), exist_ok=True)
        cv2.imwrite(SavePath, WarpedImg)
```

Python 制表符宽度：8 第 1 行，第 1 列 插入

可惜的是去年的内外参并**不太适用**于今年的视频（从场景不同也能大概猜出来）如下图，同样的参数矩阵对于去年的数据效果较好，映射后**基本为俯视图**，但对于今年的视频图片的效果就难以令人满意





后来又有同学在交流群里询问内参矩阵相关问题，我们拿到**新的内参矩阵**又进行了测试：

哈工大威海创新平台 - 718-王硕(22969079)

cameraParams | cameraParams.IntrinsicMatrix

cameraParams.IntrinsicMatrix

	1	2	3	4	5
1	308.9186	0	0		
2	0	306.9514	0		
3	325.9737	224.4464	1		

cameraParams

1x1 cameraParameters

属性	值
ImageSize	[480,640]
RadialDistortion	[-0.3314,0.1060]
TangentialDistortion	[0,0]
WorldPoints	*88x2 double*
WorldUnits	'millimeters'
EstimateSkew	0
NumRadialDistortionCoefficients	2
EstimateTangentialDistortion	0
TranslationVectors	*21x3 double*
ReprojectionErrors	*88x2x21 double*
DetectedKeypoints	*88x21 logical*
RotationVectors	*21x3 double*
NumPatterns	21
Intrinsics	*1x1 cameraIntrin...*
IntrinsicMatrix	[308.9186,0,0;0,3...
FocalLength	[308.9186,306.95...
PrincipalPoint	[325.9737,224.44...
Skew	0
MeanReprojectionError	0.2185
ReprojectedPoints	*88x2x21 double*
RotationMatrices	*3x3x21 double*

径向畸变 k1、k2 (k3为0)

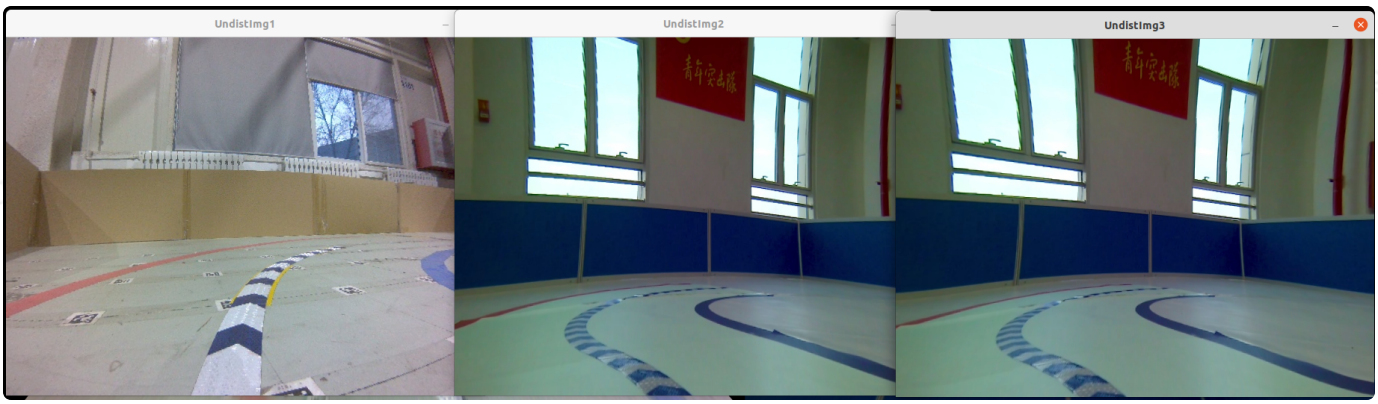
切向畸变 p1、p2

内参矩阵 (双击查看完整矩阵)

新的内参矩阵效果也并不是很理想

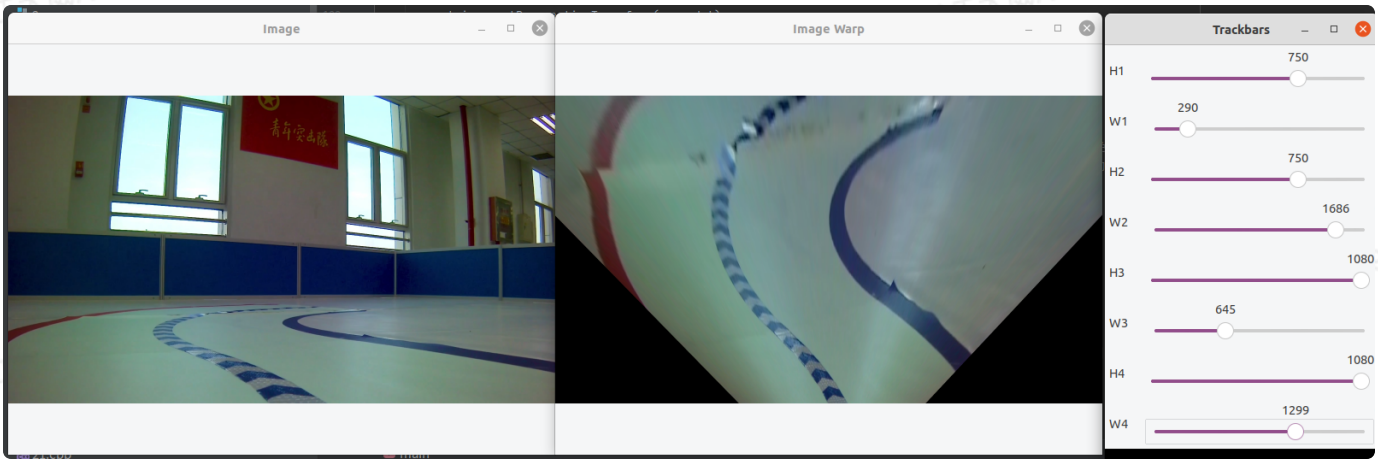
下图从左至右：**去年的图片，去年的参数；今年的图片，去年的参数；今年的图片，今年的参数**

测试各参数矩阵的程序在工程文件中的**2.cpp**



于是我们取消了使用内参矩阵去畸变的操作，对于透视变换，我们选择用trackbar来模拟一下，

trackbar模拟：



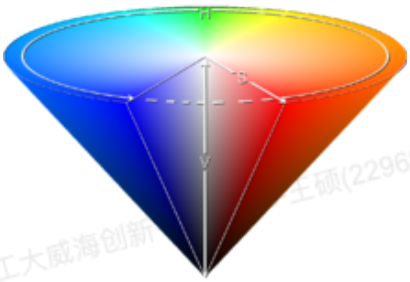
如上图，选择将原图 `{290,  750}, {1686, 750}, {0,    1080}, {1920, 1080}` 四点包围的矩形透视变换到 `{0.0, 0.f}, {1920, 0.f}, {645,  1080}, {1299, 1080}` ，看起来比较像俯视图，且完成了**ROI**区域选取，得到参数后此组参数就**固定**用于整个视频，矩阵初始化就直接写在while循环外以加快速度

使用trackbar来模拟外参矩阵映射的程序在工程文件中的**3.cpp**

Warp　　C++　　复制代码

```
////////////////////////////写在while外/////////////////////////
        Point2f src[4] = {{290,  750},
                          {1686, 750},
                          {0,    1080},
                          {1920, 1080}};//Point2f表示浮点数的点数据
        Point2f dst[4] = {{0.0f, 0.0f},
                          {1920, 0.0f},
                          {645,  1080},
                          {1299, 1080}};//Point2f表示浮点数的点数据
        matrix = getPerspectiveTransform(src, dst);
////////////////////////////////////////////////////////////////
warpPerspective(frame, imgWarp, matrix, Point(1920, 1080));
```

3.3 HSV转换



相较于RGB空间，HSV在用于指定颜色分割时，有比较大的作用。由于**亮度**与**色相**、**饱和度**的**分离**，我们仅需**H**和**S**分量就可以代表色彩信息，对于不同的彩色区域，混合**H**与**S**变量，划定阈值，即可进行简单的分割，而对于亮度**并不敏感**。

因此我们选择转换图像到HSV空间，在其中查找颜色更加容易，且不易受光照影响

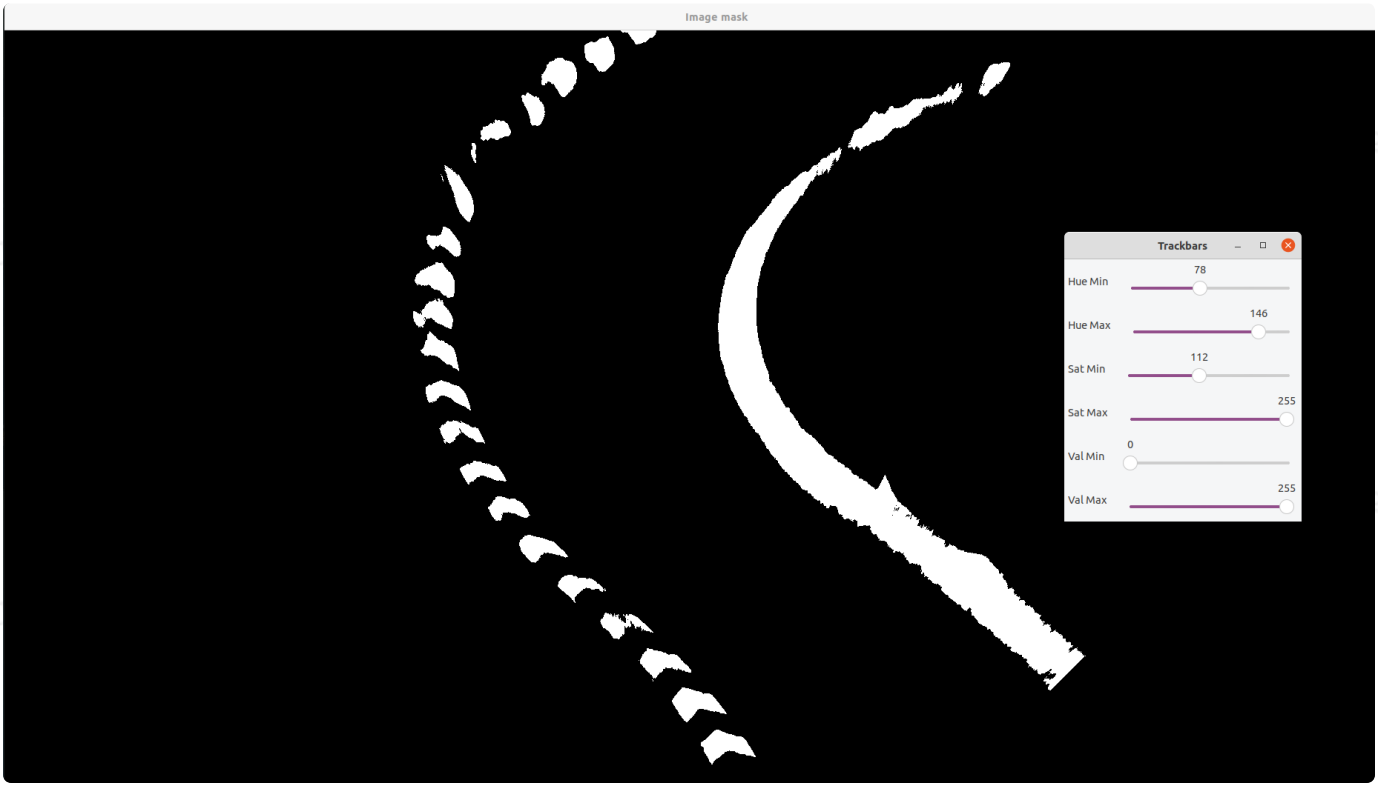
C++ 复制代码

```
1    cvtColor(imgWarp, HSV, COLOR_BGR2HSV);
```

3.4 HSV选通

同样使用**trackbar**来进行模拟，发现 `hmin=78; hmax=147; smin=112; smax=255; vmin=0; vmax=255;` 选通的效果较好，于是使用这组数据作为模板来创建蒙板进行选通，如下图；数据同样写在主函数外，方便调参

使用trackbar来筛选HSV范围的程序在工程文件中的**4.cpp**



哈工大威海创新平台 - 718-王硕(22969079)

设定HSV范围 C++ 复制代码

```cpp
//////////////////////////写在主函数外//////////////////////
int hmin=78;
int hmax=147;
int smin=112;
int smax=255;
int vmin=0;
int vmax=255;
/////////////////////////////////////////////////////////////
    Scalar lower(hmin, smin, vmin);
    Scalar upper(hmax, smax, vmax);
    inRange(HSV, lower, upper, mask);//定义颜色下限和上限，因为由于照明和不同的阴影，颜色的值将不完全相同，会是一个值的范围
```

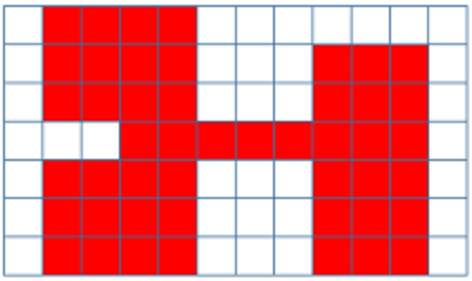
3.5 平滑边缘

可以看到上图中处理过的蒙版中轮廓**边缘较为粗糙**，且其中有**小空洞**，进行两次**闭操作**来平滑边缘和消除空洞，所谓**闭操作**，即为先进行**膨胀**操作，后进行**腐蚀**操作，关于**腐蚀、膨胀、开操作、闭操作**的效果请看如下示意图：

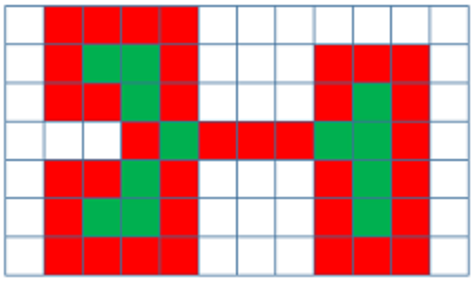




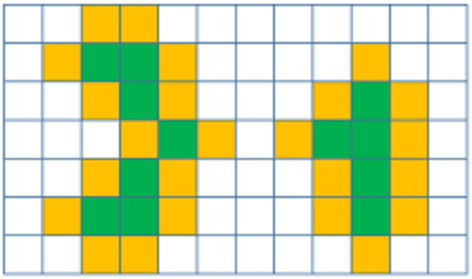
先腐蚀后膨胀的操作称之为开操作。它具有**消除细小物体，在纤细处分离物体和平滑较大物体边界**的作用



开操作前

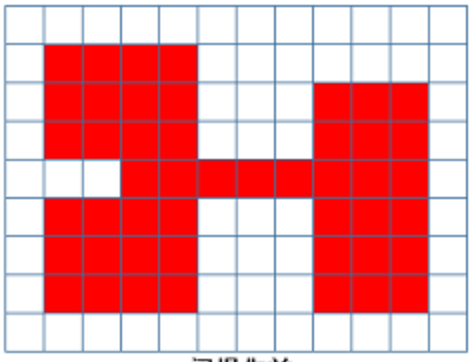


开操作-腐蚀操作（绿色）

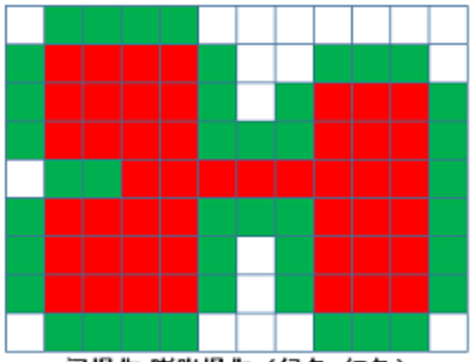


开操作-膨胀操作（绿色+黄色）

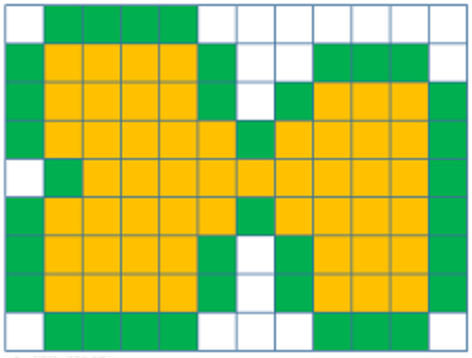
先膨胀后腐蚀的操作称之为闭操作。它具有**填充物体内细小空洞，连接邻近物体和平滑边界**的作用



闭操作前



闭操作-膨胀操作（绿色+红色）



闭操作-腐蚀操作（黄色）

合理利用闭操作，不仅可以增加边界平滑度，还可以**填充**蒙版轮廓中的**小空洞**

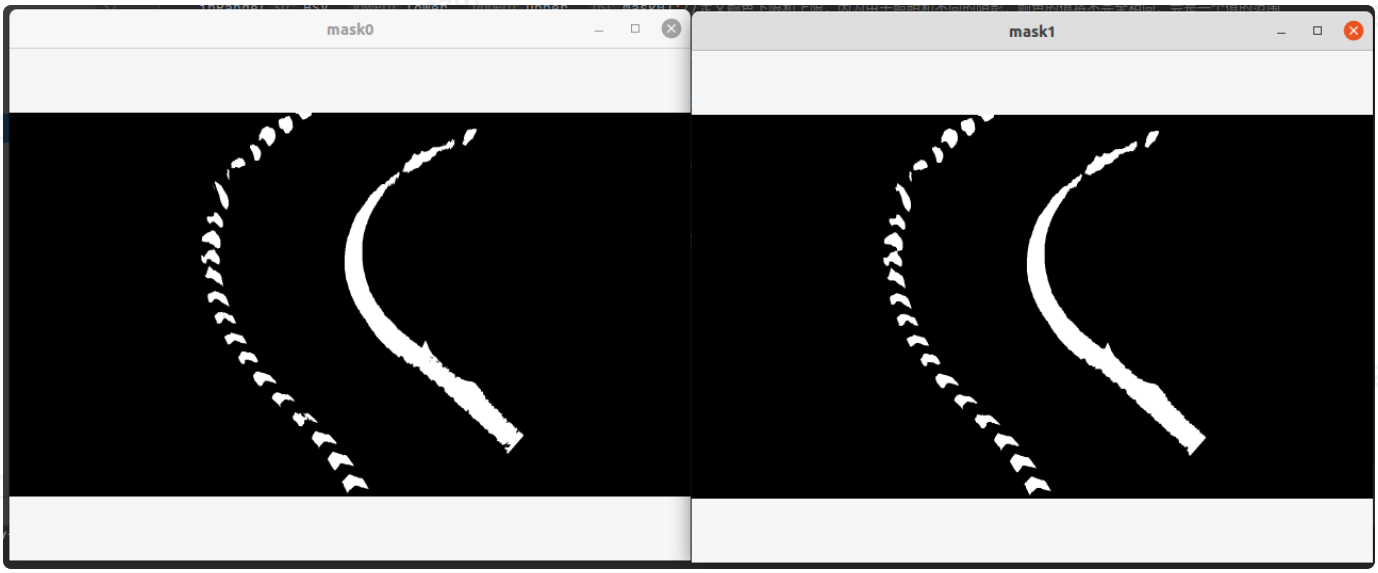
哈工大威海创新平台 - 718-王硕(22969079)

两次闭操作 C++ 复制代码

```cpp
Mat kernel = getStructuringElement(MORPH_RECT, Size(7, 7));//创建一个核，增加Size（只能是奇数）会扩张/侵蚀更多
dilate(mask, mask, kernel);//扩张边缘（增加边缘厚度）
erode(mask, mask, kernel);//侵蚀边缘（减小边缘厚度）
dilate(mask, mask, kernel);//扩张边缘（增加边缘厚度）
erode(mask, mask, kernel);//侵蚀边缘（减小边缘厚度）
```

闭操作的效果图如下：

对比闭操作前后mask边缘的程序在工程文件中的**5.cpp**



3.6 筛选车道线

此部分均集成在函数Detection()中，输入为经过透视变换的原图和HSV选通和边缘平滑后的蒙版图，无返回值，直接对透视变换后的图像进行操作

3.6.1 轮廓检测

使用OpenCV自带的**findContours()函数**来检测传入的蒙板mask中的轮廓，检测出的轮廓存于**嵌套向量**contours中，contours属于 `vector<vector<Point>>` 类型，每个检测到的轮廓存储为contours中的一个**向量**，每个向量中存储本轮廓中按照检测规则检测到的点。此外还有一个 `vector<Vec4i>` 型向量hierarchy，内每一个元素的**4**个**int型**变量——**hierarchy[i][0] ~hierarchy[i][3]**，分别表示第i个轮廓的**后一个轮廓、前一个轮廓、父轮廓、内嵌轮廓**的索引编号。因此contours和hierarchy向量**容量相同**，各向量**一一对应**。如果当前轮廓没有对应的后一个轮廓、前一个轮廓、父轮廓或内嵌轮廓的话，则**hierarchy[i][0] ~hierarchy[i][3]**的相应位被设置为**默认值-1**。检测模式设为RETR_EXTERNAL，即**只检测外轮廓**，轮廓近似方法选用 **`CV_CHAIN_APPROX_SIMPLE`** ，即**只保留拐点信息**，这样可以大大降低**空间复杂度**。

```cpp
//imgDil是传入的扩张边缘的图像用来查找轮廓，img是要在其上绘制轮廓的图像
vector<vector<Point>> contours;//轮廓检测到的轮廓。每个轮廓线存储为一个向量
vector<Vec4i> hierarchy;//包含关于映像拓扑的信息  typedef Vec<int, 4> Vec4i;具有4个整数值
//在二值图像中查找轮廓
findContours(imgDil, contours, hierarchy, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
//drawContours(img, contours, -1, Scalar(255, 0, 255), 2);//img：要绘制轮廓在什么图片上，contours：要绘制的轮廓，-1定义要绘制的轮廓号（-1表示所有轮廓），Saclar表示轮廓颜色，2表示厚度
```

3.6.2 筛选有效点

3.6.2.1 按区域大小筛选

从上面的蒙板可以看出检测出的车道线和车道边界的蓝线面积相差较大，且有时会有小面积干扰，设定面积上下界可以有效滤除干扰项，此处上下界仍写在主函数外，便于调参

```cpp
findContours(imgDil, contours, hierarchy, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
for (int i = 0; i < contours.size(); i++)
{//遍历检测到的轮廓
    int area = contourArea(contours[i]);
    //cout << area << endl;
    if (area < areamax && area > areamin)
    {
        //cout << area << endl;
        ... ...
    }
}
```

需注意的是此任务视频中车道并不平而是**有褶皱**，我们为了车道的连贯性适当放宽标准为轮廓面积100–3000均可接受，正式比赛应当不会出现这种褶皱，筛选可以更精准

3.6.2.2 多边形拟合与按边数筛选

conPoly为 `vector<vector<Point>>` 类型，其大小为contours.size()，用于作为函数**approxPolyDP()**的参数，此函数用于以指定精度拟合多边形，因为车道线中线上的花纹皆为六边形，根据**边数**同样可以筛去许多干扰项

```cpp
float peri = arcLength(contours[i], true);//计算封闭轮廓周长
approxPolyDP(contours[i], conPoly[i],0.03*peri,true);//以指定的精度近似多边形曲线。第二个参数conPloy[i]存储近似的结果，是输出。
int objCor = (int)conPoly[i].size();
if (objCor >=3  && objCor <= 8)
{
    //cout << objCor << endl;
    ... ...
}
```

此处还使用了 `float peri = arcLength(contours[i], true);` 来计算封闭轮廓周长，目的是以0.03倍轮廓周长的精度来拟合多边形，此处精度选择属于工程技巧。求出各轮廓拟合的多边形后使用 `int objCor = (int)conPoly[i].size();` 来获得多边形边数，随后即可开始筛选，同样需注意的是此任务视频中车道有**褶皱**，我们为了车道的连贯性适当放宽标准为3–8边形，正式比赛应当不会出现这种褶皱，筛选可以更精准

3.6.2.3 (可选)按长宽比筛选

经过上述筛选后基本就可消除干扰项，若极端情况下仍存在较多非车道线上轮廓，由于经透视变换后的图像可看作**俯视**，故各车道线上轮廓的**长宽比**较为固定，也可用来筛去面积与边数与车道线上轮廓相似的较**狭长**干扰项，不过在实验中并未出现此类情况，此处仅作为预案，矩形型向量 `boundRect` 用来存储各轮廓外接矩形用于拟合长宽比

```cpp
//                boundRect[i]=boundingRect(conPoly[i]);//计算边界矩形
//                float aspRatio = (float)boundRect[i].width / (float)boundRect[i].height;//宽高比
//                if (aspRatio > 0 && aspRatio < 1009 )
//                {
//                  rectangle(img,boundRect[i],Scalar(0, 255, 255),2);
```

3.6.2.4 存储各轮廓重心

由于检测到的轮廓多为凹六边形，故选择传统的**外接矩形中点**可能会偏离车道线中点，而使用**色块重心**即可较精准拟合整个车道线的中点，将这些点存入前面提到的点型向量 `points` 中等待后一步筛选

```cpp
Mat tmp(contours.at(i));
Moments moment=moments(tmp, false);
if (moment.m00 != 0)//除数不能为0
{
    int x = cvRound(moment.m10 / moment.m00);//计算重心横坐标
    int y = cvRound(moment.m01 / moment.m00);//计算重心纵坐标
    points.push_back({x,y});
}
```

此处 `Moments moment=moments(tmp, false);` 以当前的轮廓创建了一个 `Moments` 变量，此类型为轮廓的特征矩，其中的成员变量满足：

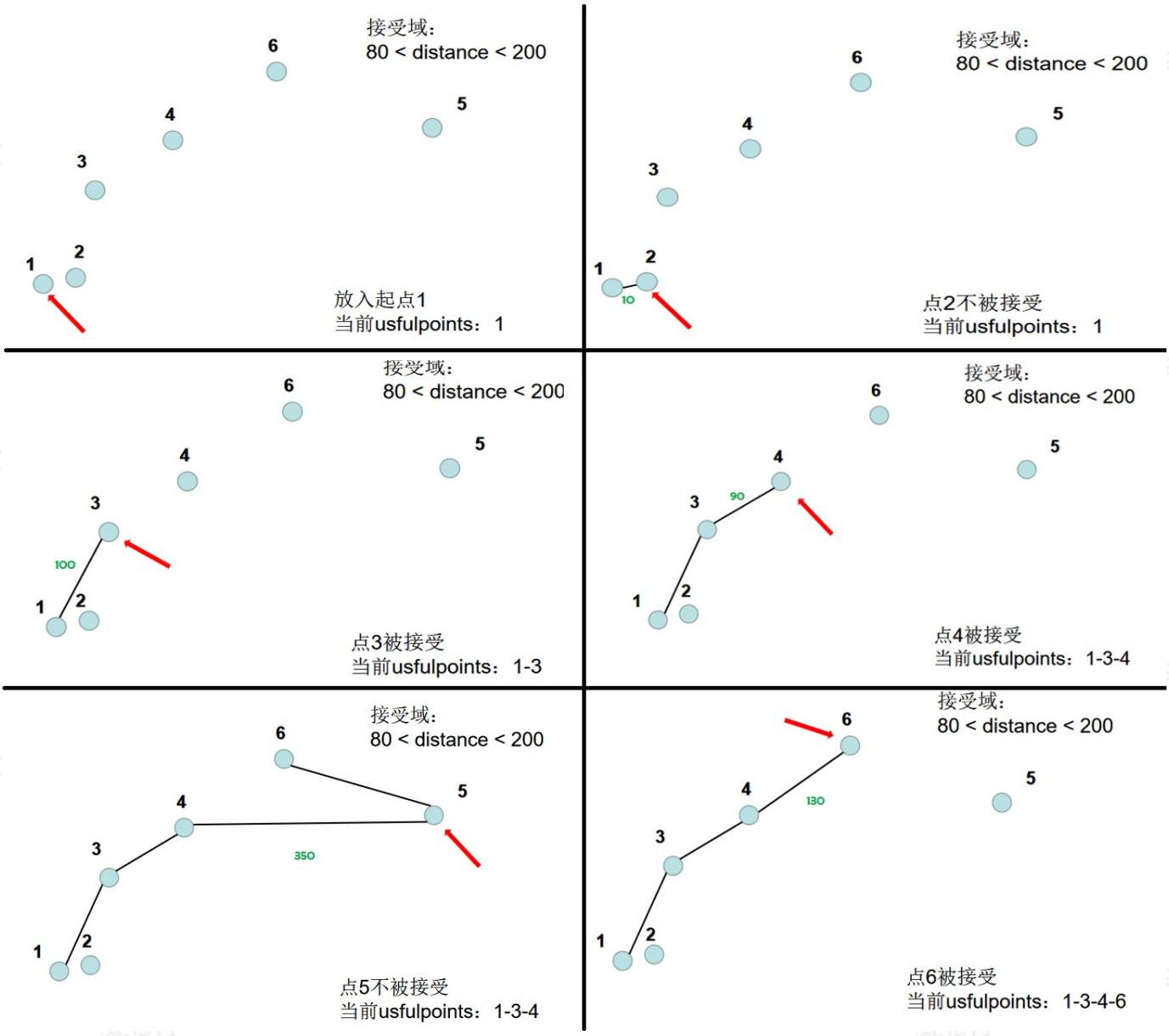
$$m_{ji} = \sum_{x,y} \left(\text{array}(x,y) \cdot x^j \cdot y^i \right)$$

对于01二值化的图像，m00即为轮廓加权面积(此处"权"指各像素点存储数据)，m10/m00即为重心的x坐标，同理，m01/m00即为重心的x坐标

3.7 连通性判定

由于车道线上各色块在俯视图中**距离较为固定**，针对此我们设计了一种以**色块重心之间距离**为特征的筛选算法来检测筛选出的各点是否属于同一条连通线，我们制作了算法的示意图：

哈工大威海创新平台 - 718-王硕(22969079)



3.7.1 寻找车道起点

一般经过上述的筛选后 `points` 中**第0个点**即可作为视线中**车道线起点**，但为提高算法鲁棒性，此处再添加一步验证来确定最终需要输出的车道线中点集 `usfulpoints` 中的首个元素

C++ | 复制代码

```
//增大找到车道起点的鲁棒性
if(dist({960,1080},points[0])<200)
    usfulpoints.push_back(points[0]);
else if (dist({960,1080},points[1])<200)
    usfulpoints.push_back(points[1]);
else
    usfulpoints.push_back(points[2]);
```

此处的 `Distance1` 同样是放在主函数外的可调参数，dist()为定义在主函数外的求两点间；距离的函数：

C++ 复制代码

```
double dist(Point p1,Point p2)
{
    return(sqrt((p2.x - p1.x) * (p2.x - p1.x) + (p2.y - p1.y) * (p2.y - p1.y)));
}
```

由于需要事先调参， `points` 中前三个元素均不能作为车道线起点的可能性极小，可以忽略，故此项筛选合理

3.7.2 迭代剔除二维数据离群点

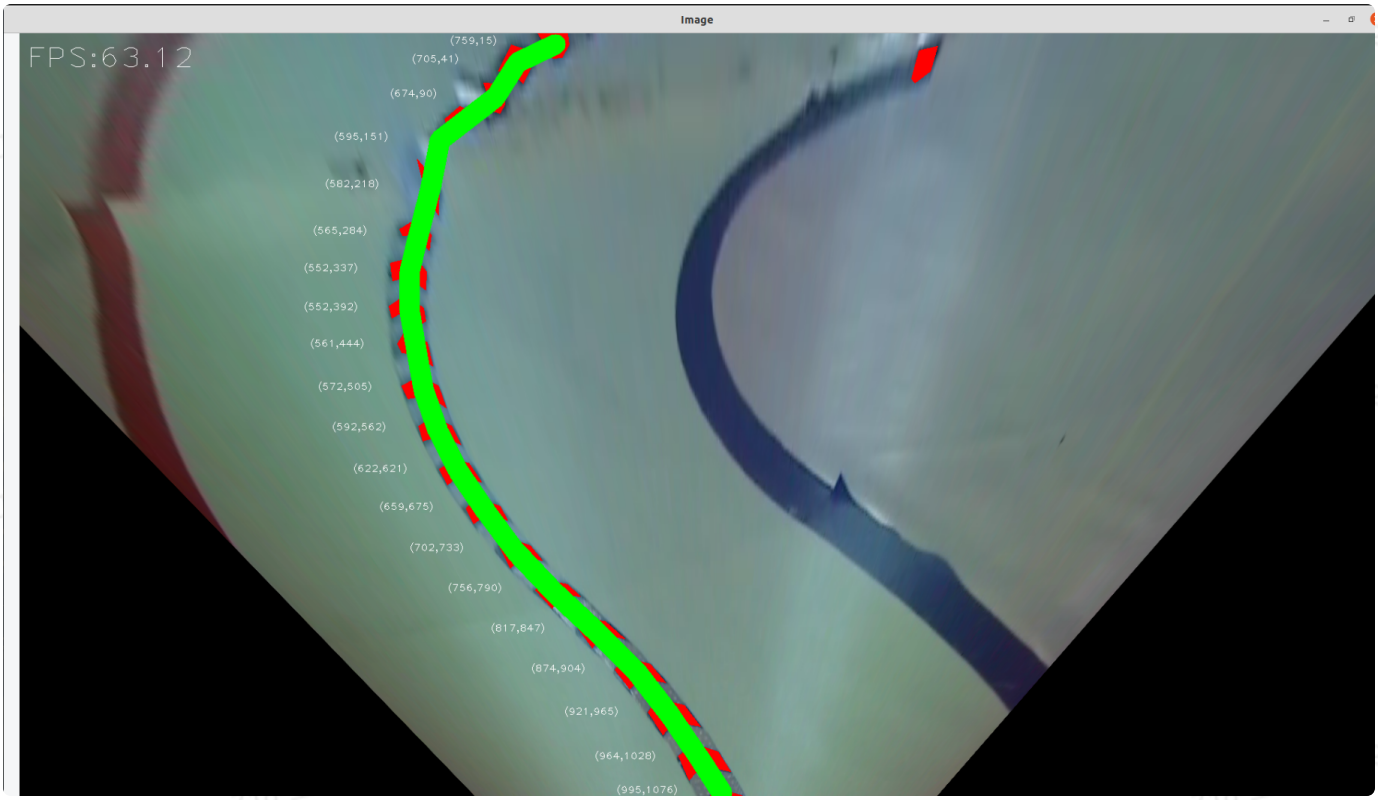
如上述算法，为了消除离群点，在确认了点集 `usfulpoints` 中的首个元素后，我们比较点集 `points` 中各点和点集 `usfulpoints` 中**末尾元素**的距离，若满足某一范围，则将点集 `points` 中的此点**压入点集** `usfulpoints` 中**作为点集** `usfulpoints` **中末尾元素**以待下次比较

C++ 复制代码

```
//剔除二维数据离群点
for(int i=0; i<points.size(); i++)
{
    //cout<<usfulpoints[usfulpoints.size()-1].x<<"   "<<usfulpoints[usfulpoints.size(s)-1].y<<"   "<<points[i].x<<"   "<<points[i].y<<"   "<<dist(usfulpoints[usfulpoints.size()-1].x,usfulpoints[usfulpoints.size()-1].y,points[i].x,points[i].y)<<endl;
    double Dist = dist(usfulpoints[usfulpoints.size()-1],points[i]);
    if(Dist < Distance1 && Dist > Distance2)
    {
        //cout<<usfulpoints[usfulpoints.size()-1].x<<"   "<<usfulpoints[usfulpoints.size()-1].y<<"   "<<points[i].x<<"   "<<points[i].y<<"   "<<dist(usfulpoints[usfulpoints.size()-1].x,usfulpoints[usfulpoints.size()-1].y,points[i].x,points[i].y)<<endl;
        usfulpoints.push_back(points[i]);
    }
```

由于采用了一个较宽泛的轮廓面积，褶皱处原来一个完整轮廓可能被识别成**多个小轮廓**，所以此处同样设置一个**距离下界**防止褶皱处识别多个点影响拟合车道线的平滑度

效果图如下：



红色快为经过了多边形拟合筛选的色块，与points中的各重心**一一对应**，可以看到视野内的所有车道上色块均被识别，且非车道线上的色块由于**距离车道线较远**而被**剔除**

3.8 展示输出

对于线下赛判定车道线趋势来说，使用透视变换后的俯视图视角的坐标点更有利于拟合车道线的方程，故展示时仍使用透视变换后的图像和坐标

输出车道线中点坐标

直接在原图像上进行覆盖字符

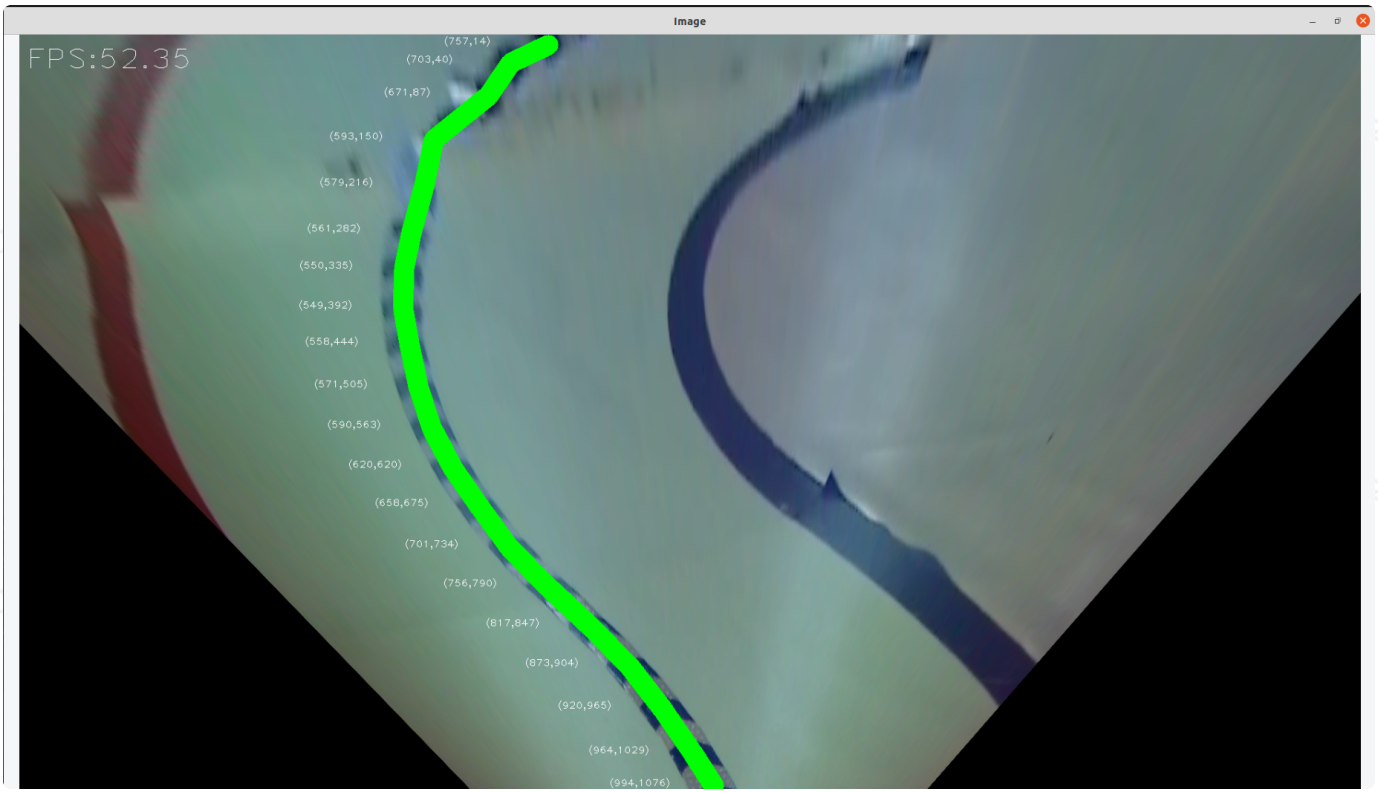
哈工大威海创新平台 - 718-王硕(22969079)

C++ 复制代码

```cpp
//输出较近距离中点坐标
for(int i=0;i<3;i++)
{
    string core;
    core="("+to_string(usfulpoints[i].x)+","+to_string(usfulpoints[i].y)+")";
    putText(img, core, Point(usfulpoints[i].x-100, usfulpoints[i].y), FONT_HERSHEY_PLAIN, 1,Scalar(255, 255, 255));
}
```

绘制车道线

以一个较粗的线连接车道线各中点即可模拟出车道线



4.创新点分析

1. 使用HSV作为色彩空间，最大程度避免光照亮度对检测结果的影响
2. 使用闭操作平滑mask蒙板，防止HSV选通后色块内部存在空洞或色块分裂
3. 使用区域面积、多边形拟合等多种方式筛选或作为预案
4. 使用Moments变量获取色块重心使其更接近车道线中点
5. 使用迭代方法维护车道线中点点集剔除二维离群点

5. 成果展示

链接：https://pan.baidu.com/s/1c6tfuOLXjUe5mlYe_OWCLQ?pwd=1024

提取码：1024

视频中实现了以下内容：

- 展示经过**透视变换**、**HSV选通**和**闭运算**后的mask
- 展示经过**轮廓检测和面积筛选**后的结果
- 展示经过**多边形拟合筛选**后的结果
- 展示经过**连通性判定筛选**后的结果**(最终结果)**
- 展示**同一组参数**对**另一个部署视频**的检测结果